DIGITAL SIGNAL PROCESSING · PROJECT DOCUMENTATION

MaskSim Explorer
The “How it works” page, explained in full

A complete walk-through of every section, every visualisation and every interactive control on the explainer page — what it shows, the mathematics behind it, what moving each slider actually changes, and what to say about it out loud.

Companion to: MaskSim — Detection of synthetic images by masked spectrum similarity analysis
Li, Bammey, Gardella, Nikoukhah, Morel, Colom & Grompone von Gioi · CVPRW 2024
Application: `webapp/static/learn.html` · `learn.js` · `lab.js` · `verdict.js`

Contents

1	What this page is, and the one idea behind it
2	Architecture — what runs in the browser, what runs on the server
3	The source bar — choosing the image the whole page works on
4	The vocabulary panel
5	Step 1 — One pair of dots = one stripe pattern (2 sliders)
6	Step 2 — Your image in the frequency domain (hover probe, lock, 2 tabs)
7	Step 3 — Your image is a sum of stripes (1 slider)
8	Step 4 — Content in the middle, fingerprints outside (1 slider)
9	Step 5 — The artifact you cannot see (2 sliders + blind test)
10	Step 6 — Your image through the real detector
11	Step 7 — The detector lab (4 sources, 1 slider, 8 stages)
12	The closing panels — key numbers and limitations
A	All the mathematics in one place
B	Control reference — every interactive element in one table
C	What is real and what is simulated
D	A six-minute demonstration script
E	Questions a examiner is likely to ask, with answers

How to open the page

```
cd C:\Users\aakar\DSPproject
.\.venv\Scripts\Activate.ps1
python -m pip install -r webapp\requirements.txt
python webapp\server.py          # wait for "Open http://127.0.0.1:8000"
```

Then open `http://127.0.0.1:8000/learn`. The server loads all seven trained detectors at start-up, which takes 20–60 seconds on a CPU and prints one line per detector. Use `python -m pip` rather than bare `pip`: this virtual environment was created while the folder had a different name, so `pip.exe` still points at the old path.

1 · What this page is, and the one idea behind it

The detector page answers a question. The *How it works* page explains why that answer can exist at all — and it does the explaining using the reader's own picture, not a canned example.

1.1 The physical claim

A generative image model does not paint pixels one at a time. It builds a small, coarse internal representation — typically 64×64 — and then **upsamples** it, several times over, until it reaches full size. The filters that perform that upsampling are *learned weights*. They are not mathematically ideal interpolators, and, crucially, they are **the same weights for every image that model ever produces**.

An imperfect upsampling filter has a frequency response that is not flat. It slightly boosts some spatial frequencies and slightly suppresses others, and it does so in a pattern that repeats across the whole picture. The result is a faint **periodic residue** in the output — typically less than one grey level out of 255, far below anything the eye can see, and far below the image's own noise.

> **Why this is a signal-processing problem and not a computer-vision one**
>
> In the spatial domain that residue is hopeless: a whisper spread thinly across a quarter of a million pixels. The Discrete Fourier Transform changes the accounting. It gathers *everything that repeats at a given rate* into a single coefficient. A whisper spread over the whole image therefore becomes a handful of bright dots, and — because the residue comes from fixed weights rather than from the scene — those dots sit at **fixed coordinates**, identical in every image that model makes. The detector never looks at the picture. It looks at those coordinates.

1.2 What the page therefore has to demonstrate

Four things, in order, and each step of the page is responsible for exactly one of them:

1. **A point in a spectrum is a stripe pattern.** Without this, nothing that follows means anything. (Step 1)
2. **Your photograph is a sum of those stripe patterns**, and its energy is overwhelmingly concentrated near the centre of the spectrum. (Steps 2 and 3)
3. **Scene content and generator fingerprints live in different places** in the spectrum — which is what allows a single fixed mask to separate them. (Step 4)
4. **A periodic artifact far too faint to see is unmistakable in the spectrum.** (Step 5)

Steps 6 and 7 then close the loop: Step 6 runs the seven real trained detectors on the uploaded image and reports what they conclude; Step 7 builds a small working detector from scratch, live in the browser, so the mechanism behind that conclusion can be watched being assembled one stage at a time.

1.3 Why the default image is a 1978 photograph

The page ships with the standard *cameraman* test tile as its fallback image. This is a deliberate choice and worth stating explicitly, because it is the first thing an examiner may query.

It is a control, not a specimen

The cameraman tile is a photograph taken decades before any generative model existed, so it is guaranteed to contain no synthesis fingerprint. Steps 1 to 5 are teaching what a spectrum *is*. For that you want a picture with nothing hidden in it — so that when Step 5 plants a fingerprint, you know with certainty that the dots which appear in the spectrum were put there by you and not by something already present. Once the page has taught the vocabulary, uploading a real subject replaces it everywhere, and Step 6 delivers an actual verdict.

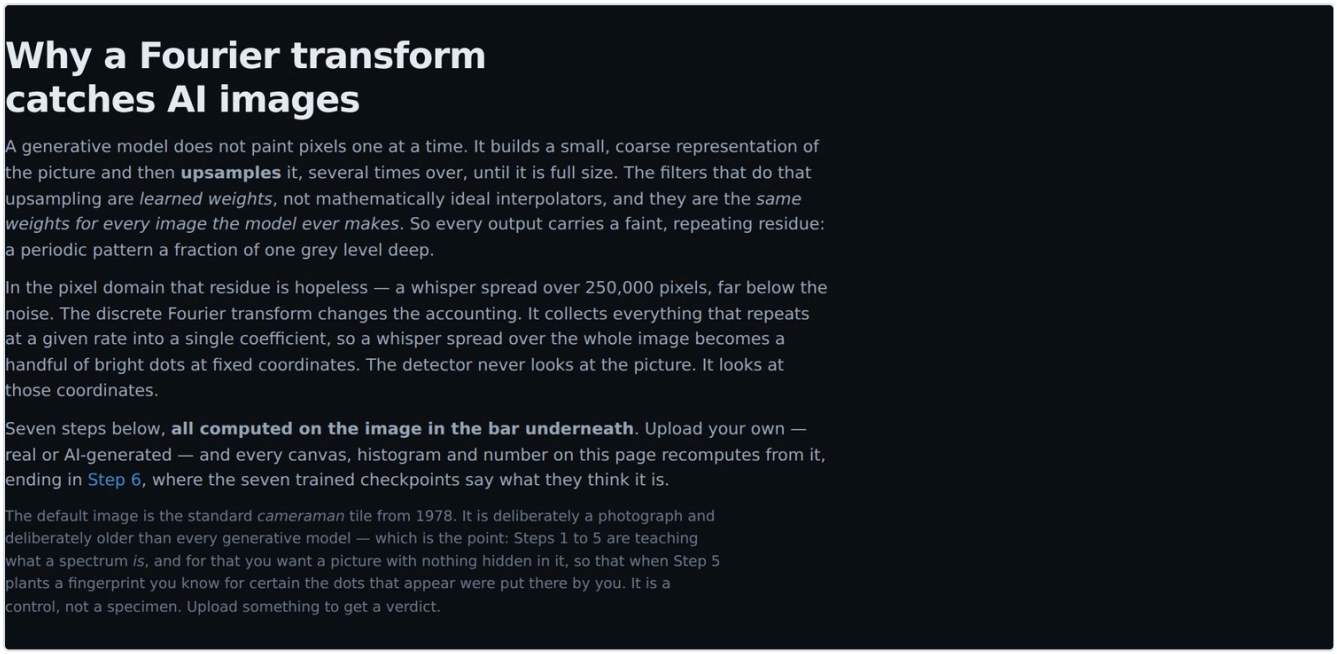


Figure 1. The opening section. The third paragraph tells the reader that everything below is computed on whatever image is in the source bar; the fourth explains the cameraman choice.

2 · Architecture — what runs where

The application has two pages that share one image. Understanding which computations happen in the browser and which happen on the server explains both the performance characteristics and the honesty claims made later.

2.1 File map

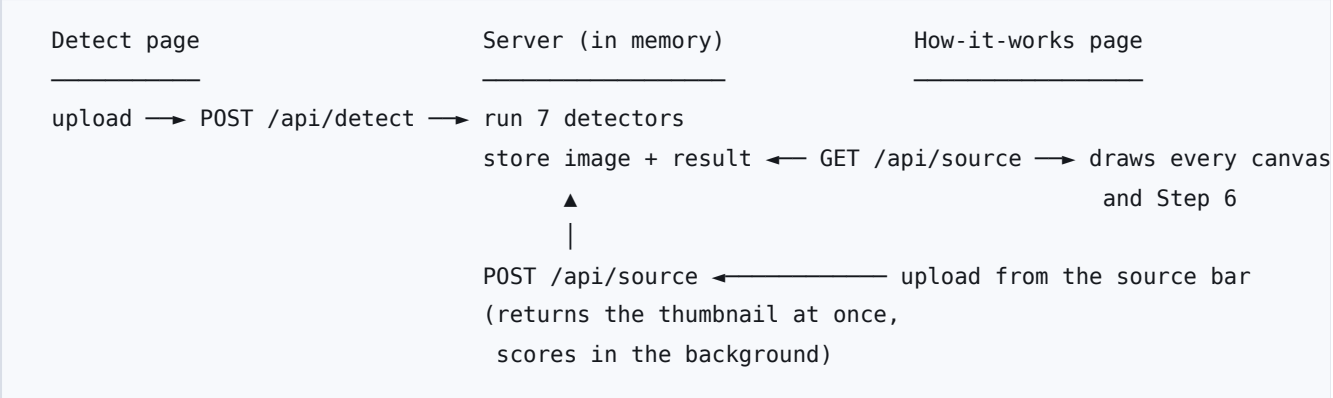
FILE	RESPONSIBILITY
`server.py`	FastAPI application. Serves the two pages and five API endpoints. Holds the "current image" that both pages share.
`analysis.py`	Wraps the original MaskSim repository. Loads the seven checkpoints, runs the real pipeline, reads the calibration tables, renders the maps as PNGs. Deliberately a *wrapper*: it re-implements nothing.
`static/learn.html`	The structure of the explainer page: all seven steps, the source bar, the vocabulary and closing panels.
`static/learn.js`	A hand-written radix-2 FFT, the canvas drawing helpers, the source bar logic, and the interactive machinery of Steps 1 to 5.
`static/verdict.js`	Step 6 — renders the seven real detectors' verdict and all their maps.
`static/lab.js`	Step 7 — builds a training set, fits a miniature detector, and drives the eight-stage walk-through.
`static/styles.css`	One dark theme shared by both pages. The categorical colour palette was checked for colour-vision deficiency.

2.2 Where each computation happens

RUNS IN	WHAT	WHY THERE
Your browser JavaScript, no libraries	Steps 1–5 and Step 7: every FFT, every spectrum, the mask fitting, the cosine scores, the calibration histogram.	Instant response to a slider. No network round-trip, so dragging feels continuous.
The server Python + PyTorch	Step 6 only: the seven trained checkpoints, the real DnCNN, the real masks, the real calibration tables of roughly 12,000 photographs each.	The checkpoints are 23 MB each and need PyTorch. This is the only part that makes a claim about your actual image.

> **The FFT is written by hand**
>
> `learn.js` contains an iterative radix-2 Cooley–Tukey FFT (`fft1`) and a separable two-dimensional wrapper (`fft2`) that transforms rows and then columns. No library is loaded. This matters for two reasons: the page works entirely offline, and — more usefully for a DSP submission — the transform at the heart of every figure is code you can open and read, about forty lines long.

2.3 How the two pages share an image



Uploading on either page updates both. The detect page gains a button reading *"Take <filename> apart in How it works"*; the explainer picks the image up automatically on load. Uploading from the explainer returns the moment the thumbnail is ready — so every canvas on the page redraws immediately — and then scores the image in a background thread, which the page polls for once a second until the verdict lands in Step 6.

2.4 One design decision worth defending

> **The crop is never resized**
>
> When the server prepares the 256×256 tile the explainer works on, it takes a **native-resolution centre crop** whenever the image is at least 256 pixels on its short side, and only resamples when the image is genuinely smaller. This is not a detail. The entire lesson of the page is that resampling destroys the fingerprint; silently resizing the demonstration image would remove the very thing being demonstrated. When an image *is* too small, the source bar says so in words.

3 · The source bar

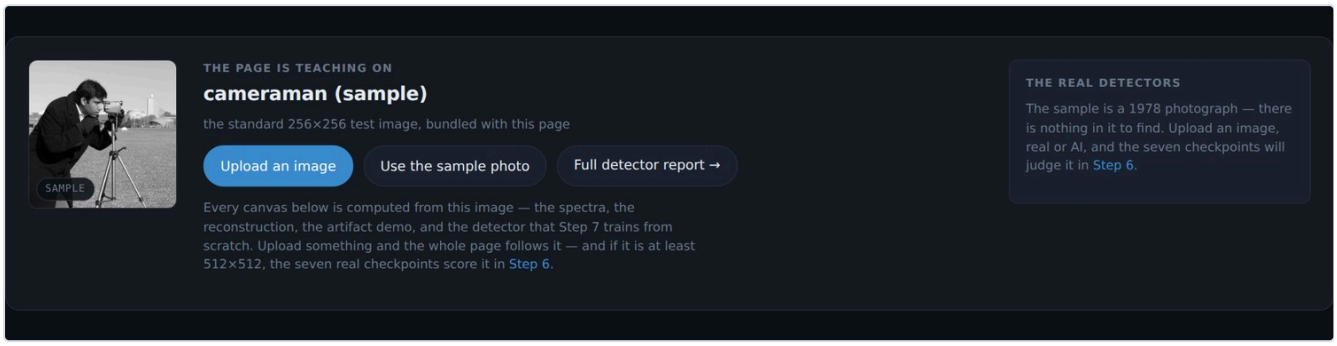


Figure 2. The source bar, shown here with an uploaded image. Left: the 256×256 tile actually used by every figure on the page. Middle: the filename, how the tile was taken, and the controls. Right: a summary of what the seven real detectors concluded, linking down to Step 6.

This bar sits directly under the introduction and governs the entire page. Everything below it — every spectrum, every histogram, the artifact demonstration, and the detector that Step 7 trains from scratch — is computed from whatever image it is showing.

3.1 The controls

CONTROL	WHAT IT DOES
Upload an image	Opens a file picker. The chosen file is POSTed to `/api/source`. The server replies as soon as it has built the 256×256 tile, so the page redraws within a fraction of a second, and then scores the image with the seven detectors in the background. A status line reports progress; Step 6 shows a spinner and then fills in.
Use the sample photo	Clears the stored image on the server and returns the page to the cameraman tile.
Full detector report →	Navigates to the detect page, which shows the same result with a few extra panels.

3.2 The three pieces of information it reports

- **The tag on the thumbnail** reads either *sample* or *your image*, so there is never any ambiguity about what the figures below are showing.
- **The provenance line** states exactly how the tile was obtained — for example *"native 256×256 centre crop of a 1920×1080 image — not resampled"*, or, for a small image, a warning that it had to be upscaled and that this weakens any fingerprint.
- **The verdict summary** on the right gives the overall conclusion, the false-positive rate, and the three most suspicious detectors, with a link to the full analysis in Step 6.

3.3 Colour convention used throughout

A point worth making to an examiner unprompted: in every chart on both pages, a result of *"no evidence"* is drawn in **grey, never green**. Green would read as "verified authentic", and that is precisely the claim this method cannot make. The three positive bands are amber (10 % false-positive rate), orange (5 %) and red (1 %).

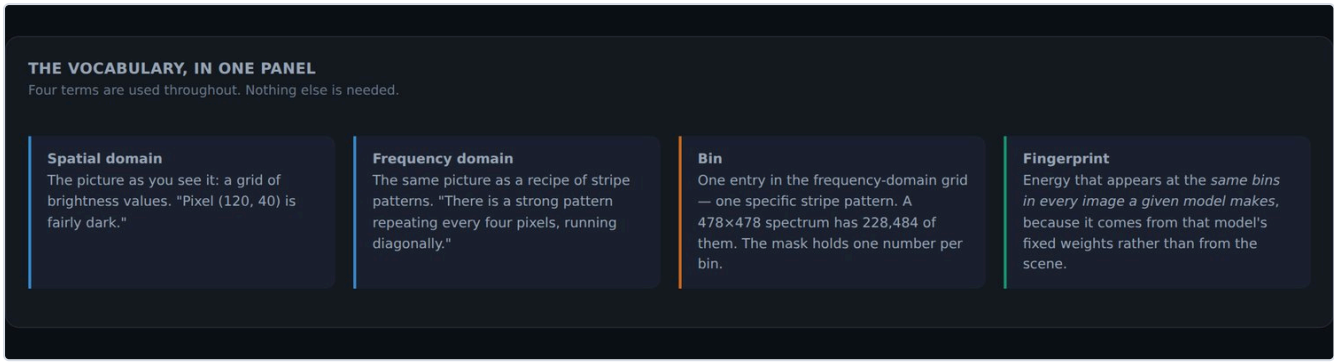


Figure 3. The vocabulary panel, immediately below the source bar. Four terms — spatial domain, frequency domain, bin, fingerprint — are all the language the rest of the page uses.

3.4 Section 4 · The vocabulary panel

This exists so that no later paragraph has to stop and define a term. The one definition that does real work later is **bin**: a single entry in the frequency-domain grid, standing for one specific stripe pattern. A 478×478 spectrum contains 228,484 of them, and the trained mask holds exactly one number per bin. Once “bin” is understood as “one stripe pattern”, the sentence “the mask opens 300 bins” becomes concrete rather than abstract.

5 · Step 1 — One pair of dots = one stripe pattern

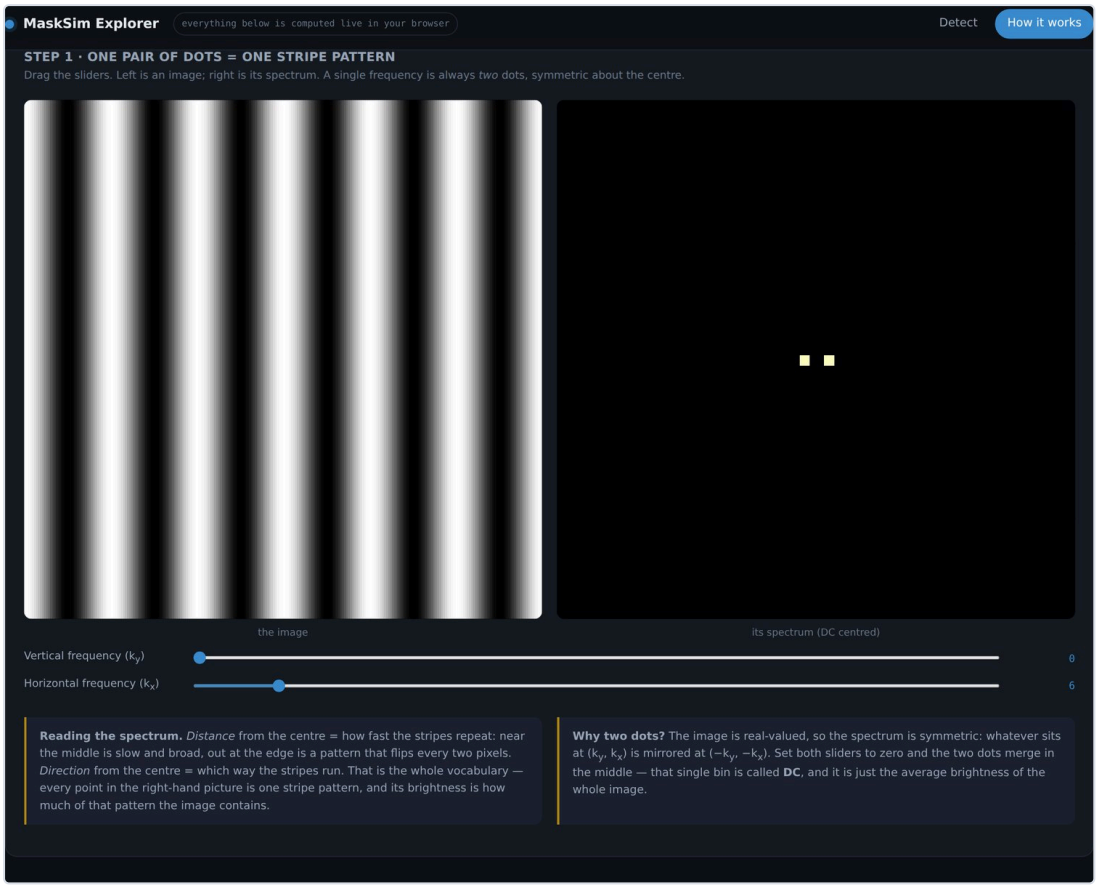


Figure 4. Step 1. Left canvas: a single cosine grating. Right canvas: its spectrum, which is exactly two dots. The two sliders set the vertical and horizontal frequency.

Purpose: to establish, with no photograph involved, that a point in a spectrum *is* a stripe pattern — and that the position of that point encodes two things, how fast the stripes repeat and which way they run.

5.1 What is being computed

The left canvas is filled with a pure two-dimensional cosine:

$$\text{image}(y, x) = \cos\left(2\pi \left(k_y \cdot y + k_x \cdot x \right) / N \right) \qquad N = 256$$

That array is passed through the same 2-D FFT used everywhere else on the page. The magnitude is taken, the logarithm applied, and the zero-frequency term shifted to the centre of the display. The result, mathematically, is two Dirac impulses at (k_y, k_x) and $(-k_y, -k_x)$.

> **One implementation detail worth mentioning**
>
> Two isolated impulses in a 256×256 image are two single pixels — effectively invisible on screen. The spectrum is therefore passed through a 5×5 **grey dilation** before display, which draws each peak as a small square. This is cosmetic and affects the picture only, never any number. The reference implementation solves the same problem the same way: `detect_one_image.py` applies `ndimage.grey_dilation(sim_map, 5)` before writing its own figures.

5.2 The two sliders

SLIDER	RANGE / DEFAULT	EFFECT
Vertical frequency k_y	`0 … 60, default 0`	Number of complete cycles the pattern makes from the top of the image to the bottom. Raising it moves both dots vertically away from the centre and tilts the stripes.
Horizontal frequency k_x	`0 … 60, default 6`	Cycles from left to right. Raising it moves both dots horizontally outwards and narrows the stripes.

Every change re-runs the whole computation — generate the cosine, FFT it, dilate, redraw — in a few milliseconds. At the default setting ($k_y = 0$, $k_x = 6$) you see six vertical stripes and two dots side by side on the horizontal axis.

5.3 The three things to demonstrate

1. **Distance from the centre = how fast.** Drag k_x from 6 to 40: the stripes get finer and the dots move outwards together. Near the centre is slow and broad; at the edge of the spectrum is a pattern that flips every two pixels — the Nyquist limit.
2. **Direction from the centre = which way.** Set $k_y = 9$ and $k_x = 3$: the stripes become diagonal, and the two dots rotate to match. The angle of the line joining the dots to the centre is exactly perpendicular to the stripes.
3. **Why two dots and not one.** The image is real-valued, so its spectrum is conjugate-symmetric: whatever appears at (k_y, k_x) is mirrored at $(-k_y, -k_x)$. Set both sliders to zero and the two dots merge in the middle. That single central bin is **DC** — the average brightness of the whole image, and nothing else.

> **WHAT TO SAY**
>
> “This is the entire vocabulary of the frequency domain. Every point in that right-hand picture is one stripe pattern like the one on the left, and its brightness is how much of that pattern the image contains. Everything later in this page is just reading that picture.”

6 · Step 2 — Your image in the frequency domain

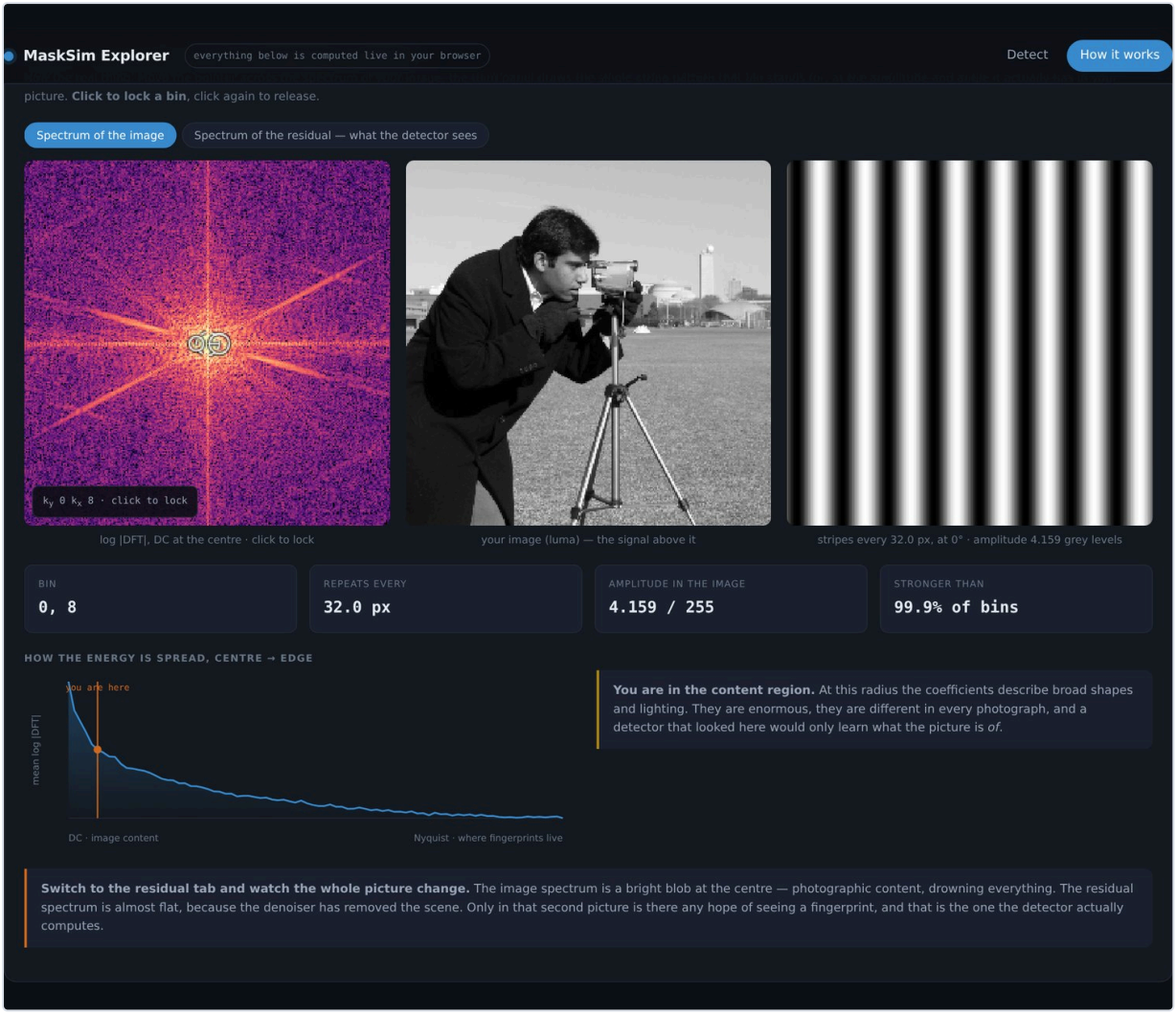


Figure 5. Step 2, the spectrum probe. Left: the log-magnitude spectrum of the current image, which responds to the pointer. Middle: the signal that spectrum was computed from. Right: the single stripe pattern the hovered bin stands for. Below: four live readouts and the radial energy profile.

Purpose: to take the abstract lesson of Step 1 and apply it to a real photograph, one frequency at a time — and, with a single tab click, to show why the detector throws the photograph away before it starts.

6.1 What is precomputed when the image loads

The moment an image is loaded, the page computes and caches two complete Fourier analyses:

TAB	SIGNAL ANALYSED	WHAT IS STORED
Spectrum of the image	The luma channel of the 256×256 tile, using the Rec. 601 weighting `0.299R + 0.587G + 0.114B`.	For each: magnitude, phase and log-magnitude at every bin (fftshifted so DC is central), plus a sorted copy of the magnitudes used for the percentile readout, plus a pre-rendered colour image of the spectrum so the crosshair can be redrawn on every mouse move without recomputing anything.
Spectrum of the residual	The same tile with its local average removed — a 3×3 box high-pass, computed with wrap-around at the edges. This is the page's stand-in for the trained DnCNN denoiser.	

6.2 The interaction

ACTION	RESULT
Move the pointer over the left canvas	The pointer position is converted from screen coordinates to a bin index, and every panel and readout updates immediately.
Click the canvas	Locks the current bin, so you can move the mouse away, point at the screen and talk. The readout changes to *locked* and the crosshair turns blue. Clicking again releases it.
Move the pointer off the canvas	If nothing is locked, the probe falls back to a default bin near the centre.
The two tabs	Switch between the image spectrum and the residual spectrum. Both were already computed, so the switch is instantaneous — and the middle panel changes too, so it always shows the signal the displayed spectrum came from.

6.3 What is drawn on the spectrum



Figure 6. A locked bin. Note the four elements of the overlay: the large circle on the selected bin, the smaller circle on its conjugate mirror, the radial line from DC, and the dashed circle marking the radius. Every stroke is drawn twice — a dark halo first, then the bright line — so the marker stays readable over both the blazing centre and the near-black corners.

- **The large circle** marks the selected bin.
- **The small circle** marks its conjugate mirror at $(-k_y, -k_x)$, reinforcing the symmetry lesson from Step 1.
- **The line from the centre** makes the direction explicit.

- **The dashed circle** is the locus of all bins at the same radius — that is, all patterns that repeat at the same rate but run in different directions. This is the visual link to the radial profile below.

6.4 The right-hand panel: one frequency, drawn honestly

This is the most valuable single panel in Step 2. It draws the cosine that the selected bin stands for, **using the amplitude and phase that frequency actually has in your image**, not a generic illustration:

```
pattern(y, x) = cos( 2π ( k_y·y + k_x·x ) / N  +  φ )

where   φ   = arg F[k]                    // the true phase of that coefficient
        amplitude = 2·|F[k]| / N²          // contribution in [0,1] brightness
units
```

The factor of 2 accounts for the conjugate pair: a real image's contribution at a given frequency is the sum of the bin and its mirror. Multiplying by 255 converts the result to grey levels, which is what the caption reports. The pattern itself is rescaled to full black-to-white for visibility — otherwise almost every choice would render as a flat grey rectangle.

6.5 The four live readouts

READOUT	FORMULA	MEANING
bin	(k_y, k_x)	The coordinates, with DC at (0, 0). Negative values are above and to the left of centre.
repeats every	$N / \sqrt{(k_y^2 + k_x^2)}$	The spatial period in pixels. A bin at radius 64 in a 256-pixel tile repeats every 4 pixels.
amplitude in the image	$255 \cdot 2\cdot\lvert F[k]\rvert / N^2$	How many grey levels of this pattern are genuinely present. Typically a few tens near DC and a few thousandths out at the edge — which is the whole problem in one number.
stronger than	rank(|F[k]|) / N² × 100	Percentile of this bin's magnitude among all 65,536, found by binary search in the sorted array. Gives a sense of whether a bin is unusually strong for its neighbourhood.

6.6 The radial energy profile

The chart under the canvases divides the spectrum into 86 concentric annuli and plots the *mean* log-magnitude in each, from DC on the left to Nyquist on the right. An orange marker labelled *you are here* tracks the radius of the probed bin.

For a photograph this curve falls away steeply and smoothly — roughly the classic `1/f` behaviour of natural images. The teaching point is the shape: almost all the energy is in the first few percent of the radius. A **bump** partway out would mean periodic structure, and that is exactly what a generator fingerprint looks like on this chart.

6.7 The adaptive commentary

The note beside the chart rewrites itself depending on where you are pointing, using the radius as a fraction of Nyquist:

RADIUS	TEXT SHOWN
`< 0.12`	*"You are in the content region."* These coefficients describe broad shapes and lighting — enormous, different in every photograph, and useless as evidence.
`0.12 – 0.45`	*"Mid frequencies: texture and edges."* Fabric, foliage, hair, JPEG blocking. Still scene-driven.
`> 0.45`	*"This is where fingerprints live."* Tiny, scene-independent coefficients. Whatever a generator leaves at a fixed coordinate out here is about the generator, not the picture. This is the region the trained mask opens.

6.8 The tab switch is the punchline

WHAT TO DEMONSTRATE

Point at the image spectrum: a single blazing blob at the centre with almost nothing elsewhere. Then click the **residual** tab. The blob vanishes and the spectrum becomes almost flat, with faint visible structure right out to the corners. The middle panel changes at the same moment, from the photograph to an edge map with the scene removed.

"The detector computes the second picture, not the first. Content is thousands of times stronger than anything a generator leaves behind, so it has to be removed before the comparison — otherwise it drowns everything. That is what the denoiser in the real pipeline is for."

7 · Step 3 — Your image is a sum of those stripes

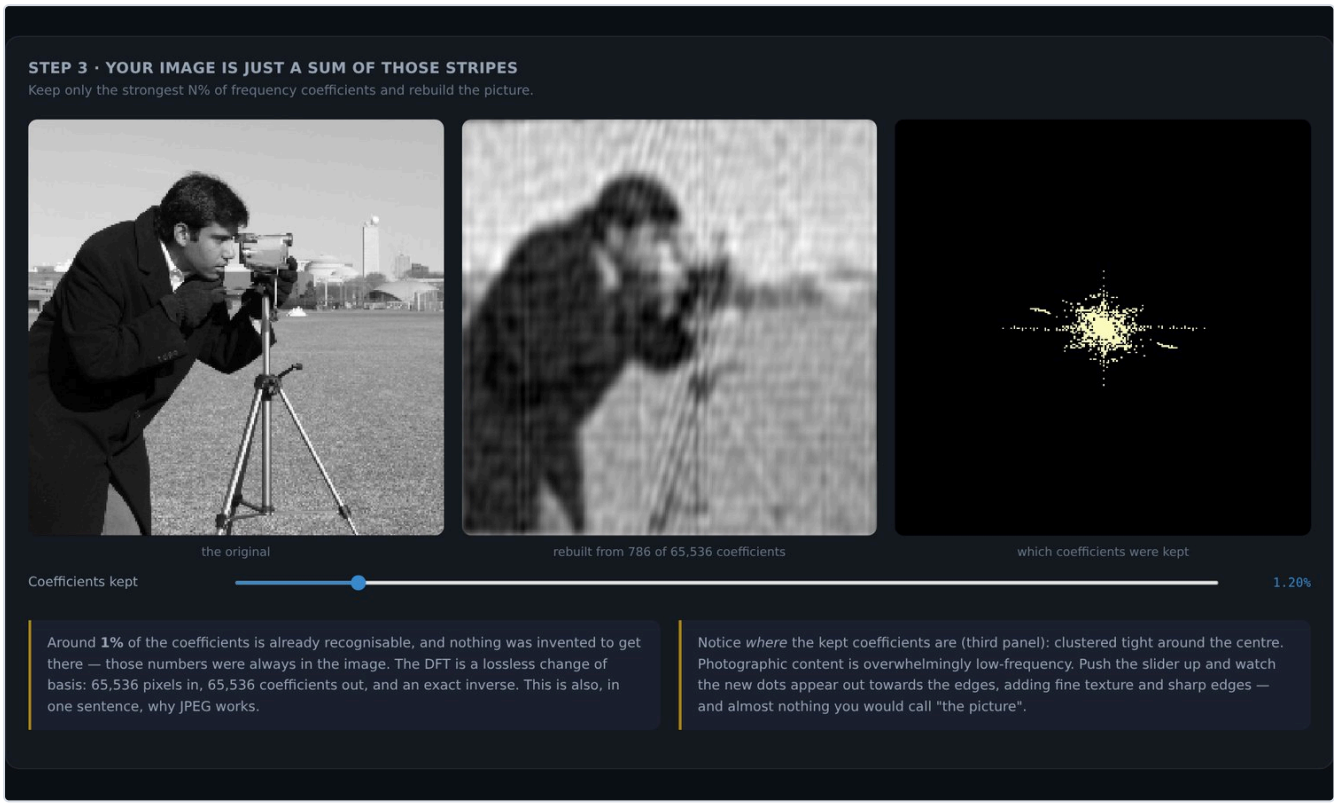


Figure 7. Step 3. Left: the original. Middle: the image rebuilt from only the strongest coefficients. Right: a binary map of which coefficients were kept.

Purpose: to prove, visually and irreversibly, that the Fourier representation contains the whole image and nothing was invented — and to show that the information is concentrated in a tiny fraction of the coefficients, clustered around the centre.

7.1 What is being computed

1. All 65,536 coefficient magnitudes are sorted.
2. A threshold is set so that exactly *n* coefficients survive, where *n* follows the slider.
3. Every coefficient below the threshold is set to zero — **both** its real and imaginary parts, so phase is preserved for the survivors.
4. An **inverse** 2-D FFT is applied and the result is drawn.
5. Separately, a binary keep-map is built and drawn, fftshifted, in the third panel.

7.2 The slider

SLIDER	RANGE / DEFAULT	EFFECT
Coefficients kept	`0.01 % … 10 %` `default 1.20 %`	Sets *n* = round(65,536 × percentage). The caption under the middle panel reports the absolute count, e.g. *“rebuilt from 786 of 65,536 coefficients”*.

7.3 The two observations

- **Around 1 % is already recognisable.** At the default setting the middle panel is clearly the same scene, slightly soft. Nothing was added to get there — those numbers were always in the image. The DFT is a lossless change of basis: 65,536 pixels in, 65,536 coefficients out, and an exact inverse. This is also, in one sentence, why JPEG works.
- **Look at where the kept coefficients are.** The third panel shows them clustered tightly around the centre. Photographic content is overwhelmingly low-frequency. Push the slider up and the new dots appear out towards the edges, adding fine texture and sharp edges — and almost nothing you would call “the picture”.

> **If asked about ringing**
>
> At very low settings the reconstruction shows ripples near strong edges. That is Gibbs ringing — the consequence of truncating a Fourier series with an abrupt cut-off — and it is the reason practical codecs use smooth quantisation rather than a hard threshold. Worth naming if the examiner points at it.

8 · Step 4 — Content in the middle, fingerprints outside

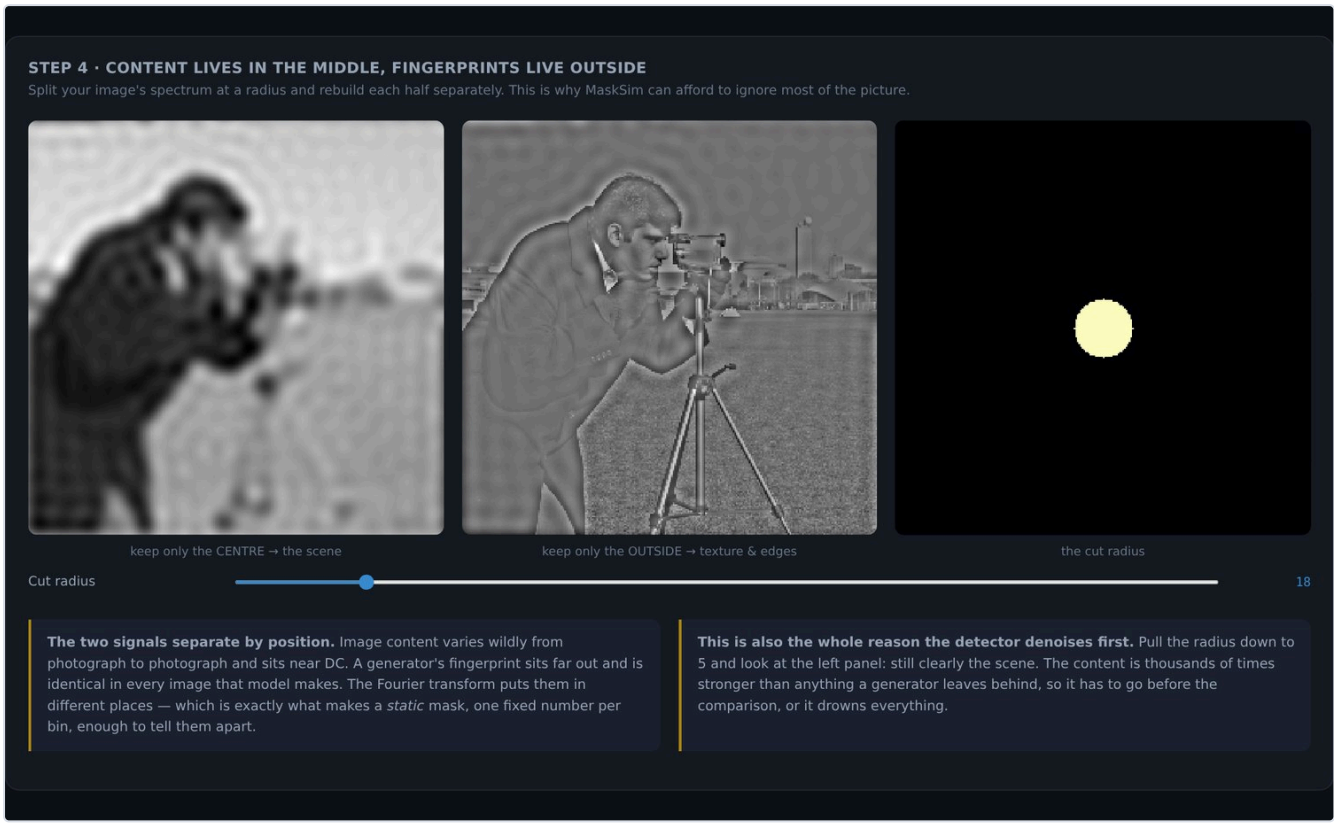


Figure 8. Step 4. The spectrum is split at a radius and each half is inverted separately: the centre alone reconstructs the scene, the outside alone reconstructs texture and edges.

Purpose: to justify the central architectural decision of MaskSim — that a single fixed mask, one number per frequency, is enough to separate a generator's fingerprint from the picture it is hiding in.

8.1 What is being computed

Every bin is classified as inside or outside a circle of radius *R* centred on DC. Two copies of the spectrum are built — one holding only the inside bins, one holding only the outside bins — and **each is inverse-transformed separately**. This is an ideal low-pass and an ideal high-pass filter, implemented in the cleanest possible way. The third panel shows the mask itself: bright inside the radius, dim outside.

8.2 The slider

SLIDER	RANGE / DEFAULT	EFFECT
Cut radius	`3 … 120 bins default 18`	The dividing circle. At 18, the left panel is a soft but complete rendition of the scene and the right panel is pure edge and texture.

8.3 The argument this step makes

> **The two signals separate by position**
>
> Image content varies wildly from photograph to photograph and sits near DC. A generator's fingerprint sits far out and is *identical in every image that model makes*. The Fourier transform puts them in different places. That is precisely what makes a static mask — one fixed number per bin, learned once and then applied to every image forever — a sufficient tool. A detector working in the pixel domain would have no comparable handle.

A second point the step makes almost by accident: pull the radius down to 5 and the left panel is still recognisably the scene. Content is not merely stronger than the fingerprint, it is stronger by orders of magnitude, and it is all crammed into a handful of bins. That is the quantitative justification for denoising first.

9 · Step 5 — The artifact you cannot see

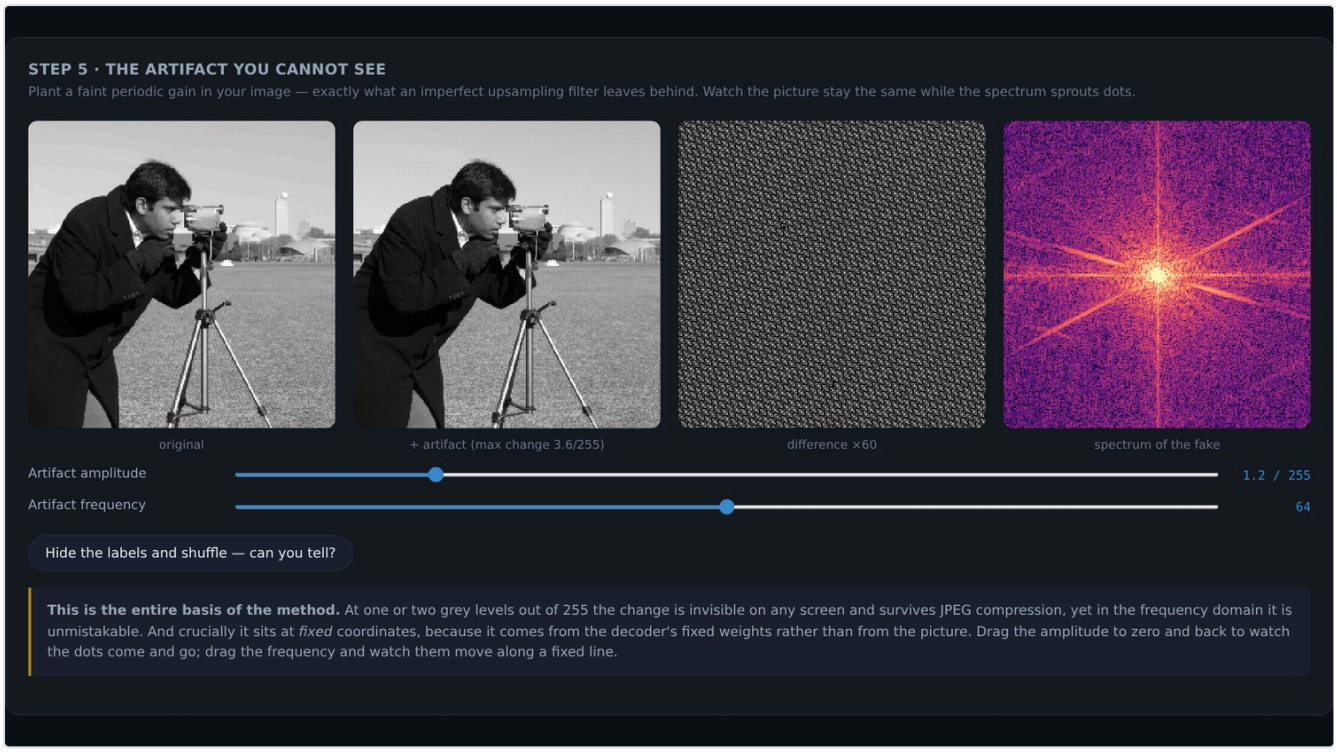


Figure 9. Step 5. Four panels: the original, the same image with a periodic artifact added, the difference amplified sixty times, and the spectrum of the modified image.

Purpose: the single most persuasive demonstration on the page. It plants an artifact in the reader's own image that is invisible on any screen, and shows it blazing in the frequency domain.

9.1 What is being computed

A periodic gain is added to every pixel. Three cosine components are used rather than one, so that the resulting spectrum shows a small constellation of peaks rather than a single pair — which is closer to what a real upsampling filter leaves:

```
g(y, x) = A · [ cos(2π f y / N) + cos(2π f x / N) + cos(2π · 0.75f · (y + x) / N) ]

fake(y, x) = clamp( image(y, x) + g(y, x),  0,  1 )
```

The four panels then show: the original; the modified image; the absolute difference multiplied by sixty so it is visible at all; and the log-magnitude spectrum of the modified image. The caption under the second panel reports the largest change any single pixel underwent, in grey levels out of 255.

9.2 The two sliders

SLIDER	RANGE / DEFAULT	EFFECT
Artifact amplitude	`0 … 6.0 / 255 default 1.2 / 255`	The coefficient *A*. At the default, the largest change to any pixel is under two grey levels — invisible. Drag to zero and the extra dots in the spectrum vanish; drag back and they return.
Artifact frequency	`8 … 120 cycles default 64`	The frequency *f*. Raising it drives the peaks outwards along a fixed line — a direct visual restatement of the Step 1 lesson, now on a real photograph.

9.3 The blind test

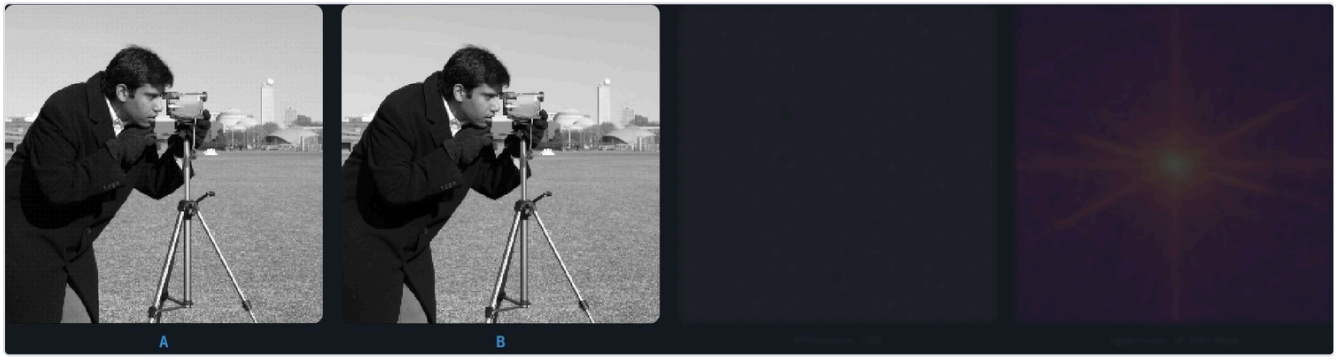


Figure 10. Blind mode. The labels are replaced with “A” and “B”, the panel order is randomised, and the difference and spectrum panels are blurred out so they cannot give the answer away.

The button under the sliders reads *“Hide the labels and shuffle — can you tell?”*. Pressing it randomises which of the two images appears on the left, relabels them A and B, and suppresses the two revealing panels. Pressing it again reveals which one carried the artifact.

WHAT TO SAY

“One of these two has a periodic fingerprint planted in it, a little over one grey level deep. Nobody in this room can tell which. Now watch —” *(reveal, then point at the spectrum panel)* “— in the frequency domain it is not subtle at all. And because it comes from the decoder's fixed weights rather than from the picture, it sits at the same coordinates in every image that model makes. That is the entire basis of the method.”

Be precise about what this proves

This step demonstrates the *detectability* of a periodic artifact, using an artifact the page itself planted. It does not prove that any particular generator leaves one. That evidence comes from Step 6, where real trained detectors examine a real image.

10 · Step 6 — Your image through the real detector

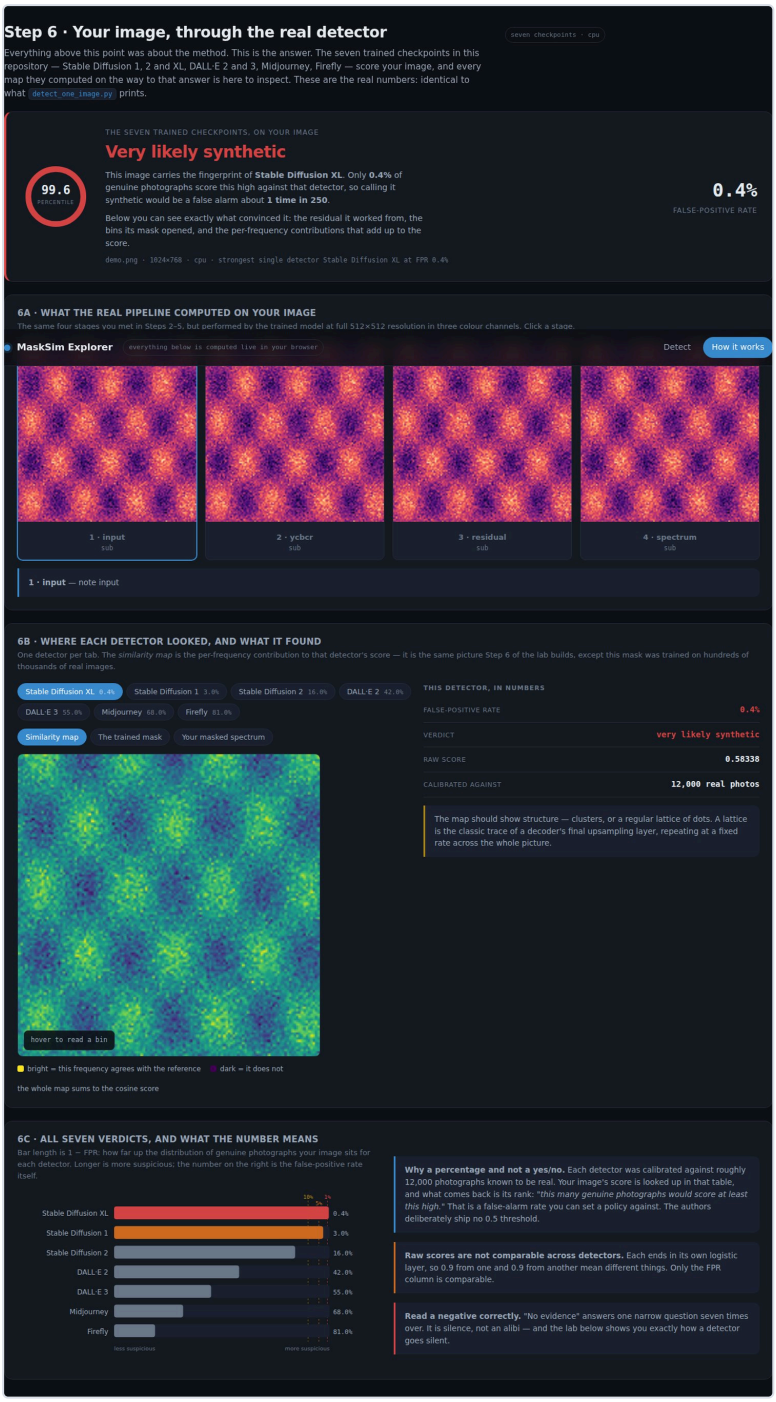


Figure 11. Step 6 with an image loaded. The verdict card, the four real pipeline stages, the per-detector map explorer, and all seven false-positive rates. (Screenshot taken against a test harness, so the numbers are placeholders; the layout and interactions are the real ones.)

Purpose: this is the only part of the page that makes a claim about your actual picture. Everything above it teaches; this concludes. The numbers here come from the seven trained checkpoints in the repository and are identical to what `detect_one_image.py` prints.

10.1 The four states of this panel

STATE	WHEN
Nothing scored	On the sample image. Explains that the cameraman is a 1970s photograph with nothing to find, and invites an upload.
Scoring	While the background thread runs the seven detectors. Shows a spinner; the page polls once a second.
Not scored	If the image is smaller than 512×512. Every other figure on the page still uses it; only this panel, which needs the full crop the checkpoints were trained on, is unavailable.
Result	The full three-part report described below.

10.2 The verdict card

A ring showing the percentile, the verdict in words, the false-positive rate as a large number, and a two-paragraph explanation that is written differently depending on the outcome:

- **If nothing matched**, the text names the closest detector, states how many genuine photographs score at least as high, and then says plainly: *"That is not a certificate of authenticity."* It explains that an image from a generator none of the seven was trained on looks exactly like this — and so does a real photograph that has been resized.
- **If something matched**, it names the generator, converts the false-positive rate into odds ("a false alarm about 1 time in 250"), and points down at the maps that produced the result.

10.3 Panel 6a — the four real pipeline stages

Four clickable thumbnails showing what the trained model actually computed on your image, at full 512×512 resolution in three colour channels: the centre crop, the luma channel, the DnCNN residual, and the log-magnitude spectrum. Clicking one writes an explanation underneath. These are the same four stages the reader met in Steps 2 to 5, now performed by the real model rather than by the browser.

10.4 Panel 6b — the per-detector map explorer

Seven tabs, one per generator, each showing its false-positive rate beside its name. Three further tabs select what is displayed:

LAYER	WHAT IT IS
Similarity map	The per-frequency contribution to that detector's score. Every pixel is one term of the cosine sum; the whole map adds up to the score. This is the same picture the command-line tool writes as `sim_map_<generator>.html`, using the same colour scale and the same per-generator range.
The trained mask	The learned weights, 478×478 per channel, squashed into [0,1]. Static — identical for every image ever scored. Worth dwelling on: this is what twenty epochs of training produced, and you can simply look at it.
Your masked spectrum	Your image's spectrum after the mask is applied. Only this reaches the comparison.

Beside the map, four numbers for the selected detector (false-positive rate, verdict, raw score, size of its calibration set) and a note that changes depending on whether that detector fired — telling you whether to expect unstructured speckle or a regular lattice of dots.

10.5 Panel 6c — all seven verdicts

A bar chart where **length is 1 − FPR**, so longer means more suspicious, with the false-positive rate itself printed at the right of each row and dashed lines at the paper's three operating points. Beside it, three explanatory notes covering the points an examiner is most likely to probe: why the output is a percentage rather than a yes/no, why raw scores cannot be compared between detectors, and how to read a negative result correctly.

10.6 The one sentence that matters most

> **Absence of evidence is not evidence of authenticity**
>
> Each detector answers one narrow question: *does this image carry this particular generator's fingerprint?* Seven “no” answers mean seven specific models left no trace. They do not mean a camera took the picture. The page states this three separate times, in the verdict text, in panel 6c and in the limitations panel, because it is the single most common way a tool like this gets misread.

11 · Step 7 — The detector lab

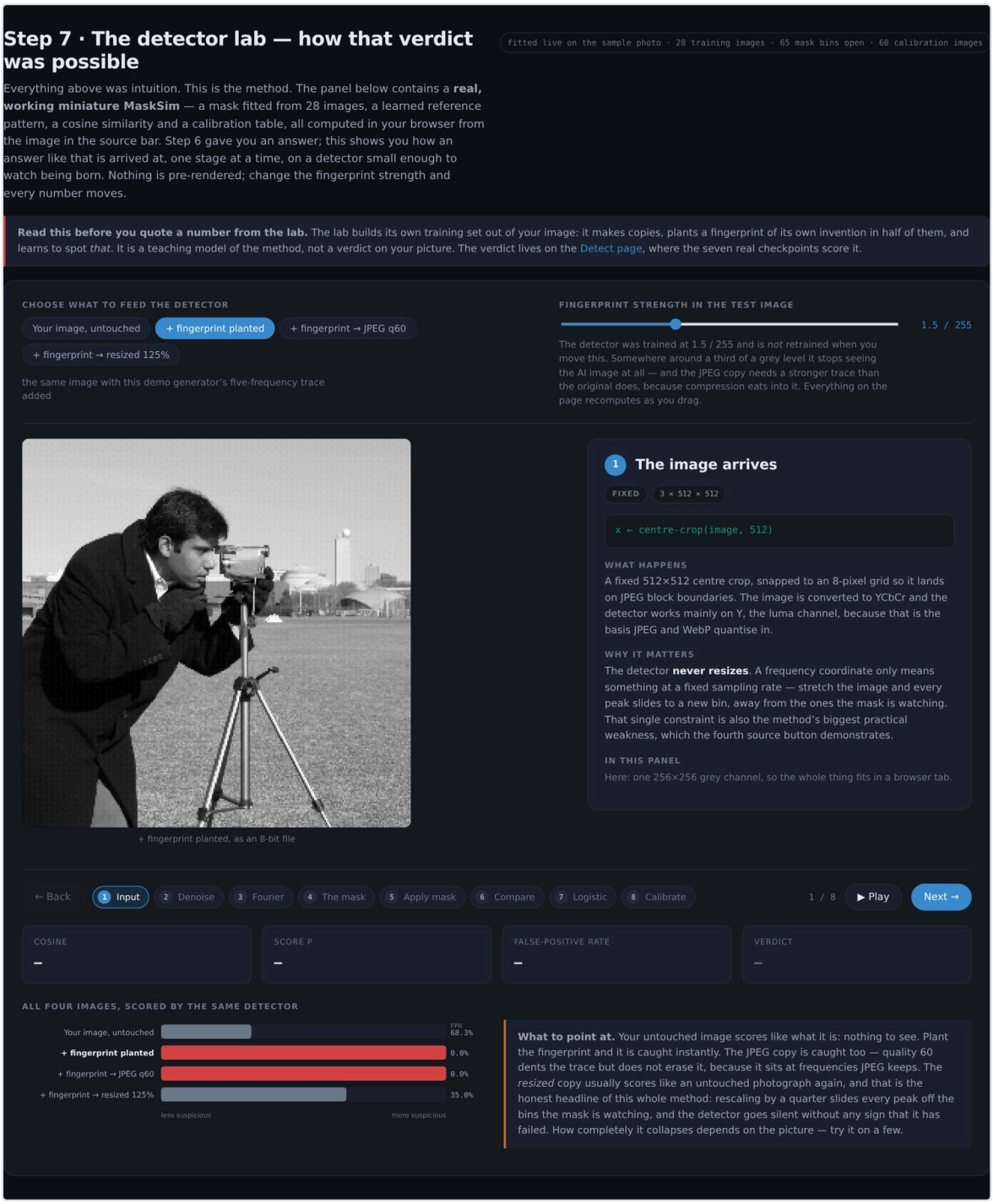


Figure 12. The detector lab. Top: the four test images and the fingerprint strength control. Middle: the current stage's visuals and its explanation. Bottom: the stage navigator, the running numbers, and a summary comparing all four images.

Purpose: Step 6 gives an answer. Step 7 shows how an answer like that becomes possible, by building a complete working detector — small enough to watch being assembled — live in the browser, from the reader's own image.

11.1 What it actually does when the page loads

This is not an animation. When the page loads (and again whenever the source image changes) the lab performs a real training run:

1. **Builds a dataset.** 28 images derived from your picture: 14 “real” variants (random circular shift, flip, gain, brightness and noise) and the same 14 with a five-frequency fingerprint planted in them. Every image is quantised to 8 bits, exactly as a real file would be, so the artifact has to survive quantisation.
2. **Computes a residual and a spectrum** for all 28.
3. **Fits the mask.** At every bin it computes the mean log-magnitude for each class, the gap between them, and the pooled standard deviation. The ratio is a per-bin signal-to-noise ratio. A threshold is set at the 99.9th percentile of that ratio and the mask is a sigmoid around it — leaving roughly 65 bins open out of 65,536.
4. **Derives the reference** as the masked difference pattern, centred and normalised to unit length.
5. **Fits the logistic** on the 28 training scores.
6. **Builds a calibration table** from 60 further “real” variants that the mask never saw. Deliberately, 45 % of those are resampled at a random scale, because a fair share of real photographs on the web have been rescaled at some point — and that is what gives the “real” distribution its realistic width.

The status badge beside the heading reports the result of the fit, e.g. *“fitted live on myphoto.jpg · 28 training images · 65 mask bins open · 60 calibration images”*.

> **State this before quoting any number from the lab**
>
> The lab plants a fingerprint **of its own invention** and learns to detect that. It is a teaching model of the method, not a verdict on your picture. The verdict is Step 6. The page carries this warning in a red-bordered box directly above the panel, and it is worth reading aloud rather than waiting to be asked.

11.2 The four test images

BUTTON	WHAT IT IS
Your image, untouched	The source image exactly as it arrived, with nothing planted in it.
+ fingerprint planted	The same image with the demo generator's five-frequency trace added.
+ fingerprint → JPEG q60	The planted version put through a genuine JPEG round-trip at quality 60, using the browser's own encoder.
+ fingerprint → resized 125 %	The planted version rescaled by 1.25 with bilinear interpolation and re-cropped, exactly as an upload site would do.

All four are scored by the same detector, and the comparison is the point of the whole panel. On a typical photograph you should see: untouched → no evidence; planted → caught outright at 0 %; JPEG → still caught; **resized → scores like an untouched photograph again**.



Figure 13. The summary chart at the foot of the lab, comparing all four images. Bar length is 1 − FPR, so longer is more suspicious. The resized copy collapses back into the harmless band.

11.3 The fingerprint-strength slider

SLIDER	RANGE / DEFAULT	EFFECT
Fingerprint strength in the test image	`0 … 4.5 / 255 default 1.5 / 255`	Rebuilds the three planted images at the chosen amplitude and rescores all four. The detector itself is **not** retrained — it stays fitted at 1.5 / 255, which is the whole point.

Dragging it down produces a graded, genuinely informative failure. Somewhere around a third of a grey level the detector stops seeing the planted image at all; and the JPEG copy needs a stronger trace than the original does, because compression eats into it. At zero, the "planted" image is identical to the untouched one and scores exactly the same — a clean demonstration that the detector is responding to the artifact and nothing else.

11.4 The eight stages

Navigate with **Back** / **Next**, the left and right arrow keys, the numbered chips, or **Play**, which advances automatically every four seconds. Each stage shows real tensors on the left and, on the right, a card with: the stage title, a badge saying whether that step is learned or fixed DSP, the tensor shape, the equation, *what happens*, *why it matters*, and a note on how this panel differs from the real model.

#	STAGE	WHAT IT SHOWS
1	**Input** fixed	The chosen image as an 8-bit file. Explains the 512×512 centre crop, the 8-pixel grid alignment onto JPEG block boundaries, and why the detector never resizes.
2	**Denoise** fixed, pre-trained	Input beside residual. The scene disappears. Explains that the denoiser predicts the *noise*, not the clean image, and why 512 becomes 478 (a 17-layer 3×3 network has a 17-pixel invalid border).
3	**Fourier** fixed	Residual beside its log-magnitude spectrum. Notes that phase is discarded: where the pattern sits does not matter, only that it repeats.
4	**The mask** learned	*The key stage.* Four panels — mean spectrum of the real class, mean spectrum of the fake class, the gap divided by its spread, and the resulting mask.
5	**Apply mask** fixed	Your spectrum × the mask = what survives. The noise-rejection step.
6	**Compare** learned	Masked spectrum, learned reference, and the per-bin contribution map that sums to the cosine. The running cosine value is printed underneath alongside the two class averages.
7	**Logistic** learned, 2 parameters	The fitted sigmoid curve with your image marked on it, and the two class averages as dashed lines.
8	**Calibrate** measured	A histogram of the 60 known-real scores with your image marked, plus the thresholds for a 10 %, 5 % and 1 % false-alarm rate, and short ticks showing where the other three test images fall.

11.5 Stage 4 in detail — the stage to spend time on

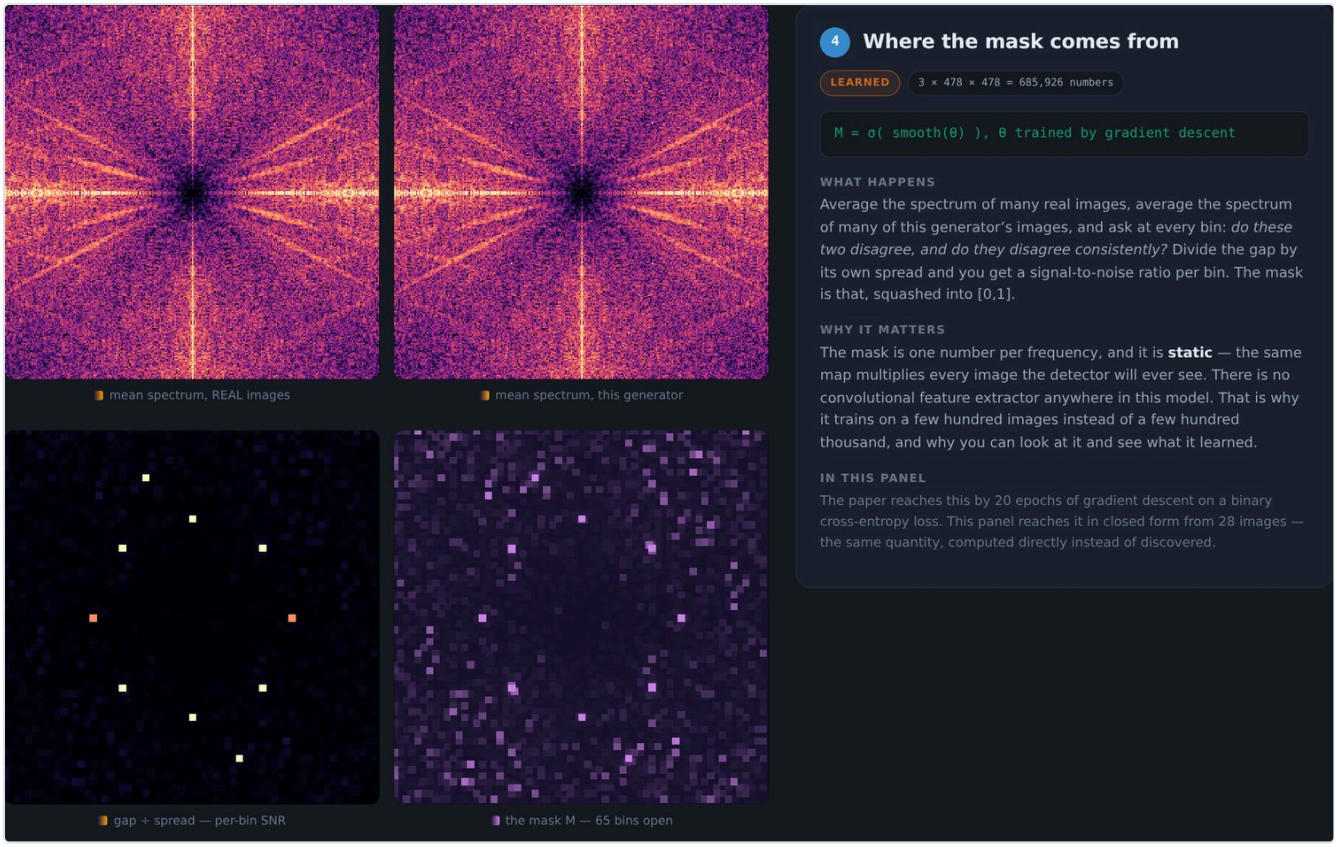


Figure 14. Stage 4. The first two panels — the mean spectra of the two classes — are indistinguishable by eye. The third, their difference divided by its own spread, reveals the ten fingerprint bins. The fourth is the resulting mask.

This is the stage that makes the method comprehensible. Average the spectrum of many real images; average the spectrum of many of this generator's images; then ask at every bin: *do these two disagree, and do they disagree consistently?*

```
z[k]  =  | mean_fake[k] − mean_real[k] |  /  pooled_std[k]

M[k]  =  σ( ( z[k] − threshold ) · 9 )        threshold = 99.9th percentile of z
```

The first two panels look identical — that is the point, and it is worth pausing on. The third panel, the signal-to-noise ratio, shows ten clean dots emerging from nothing: five frequency pairs. The fourth is the mask that results.

> **The honest difference from the paper**
>
> The published model reaches its mask by twenty epochs of gradient descent on a binary cross-entropy loss. This panel reaches it in closed form from 28 images. These are the same quantity — a per-bin measure of how reliably the two classes separate — computed directly rather than discovered by optimisation. The stage card says so in its own words, under “in this panel”.

11.6 Stage 6 and the similarity map

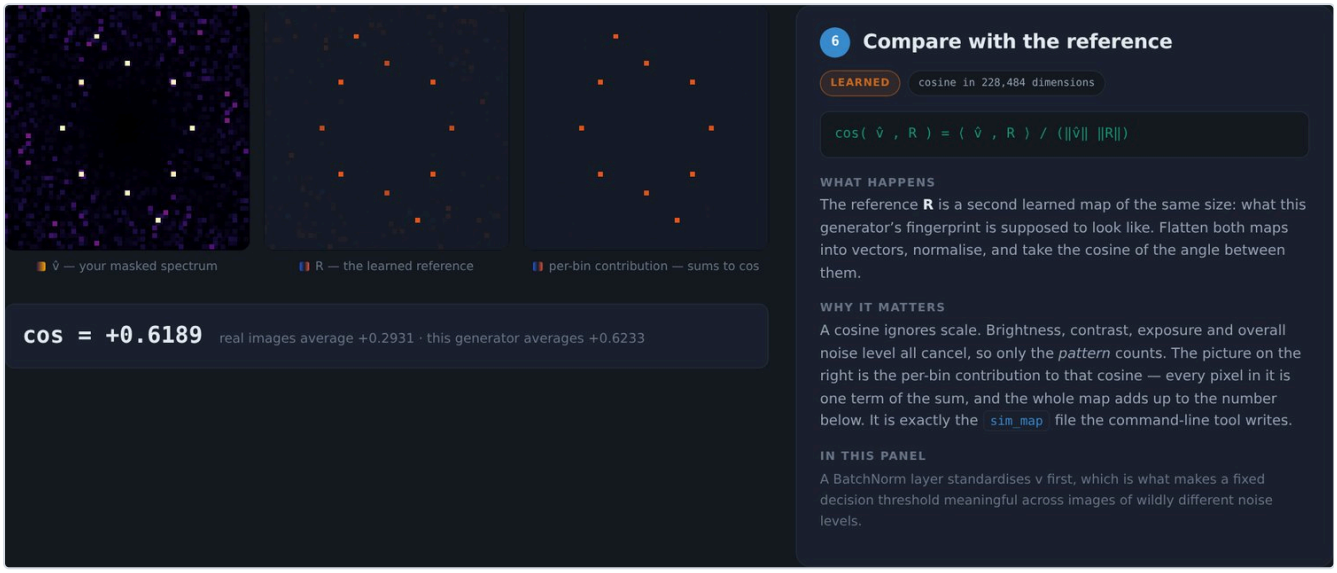


Figure 15. Stage 6. Left: your masked spectrum. Middle: the learned reference. Right: the per-bin contribution map, every pixel of which is one term of the cosine sum.

The score is a cosine similarity between the masked spectrum and the reference, taken in a space of 228,484 dimensions in the real model. A cosine is used rather than a dot product or a distance because it **ignores scale**: brightness, contrast, exposure and overall noise level all cancel, so only the *pattern* counts.

The third panel is the same object the command-line tool writes out as its `sim_map` file. Every pixel is one term of the sum, and the whole map adds up to the number printed underneath.

11.7 Stage 8 and why calibration exists

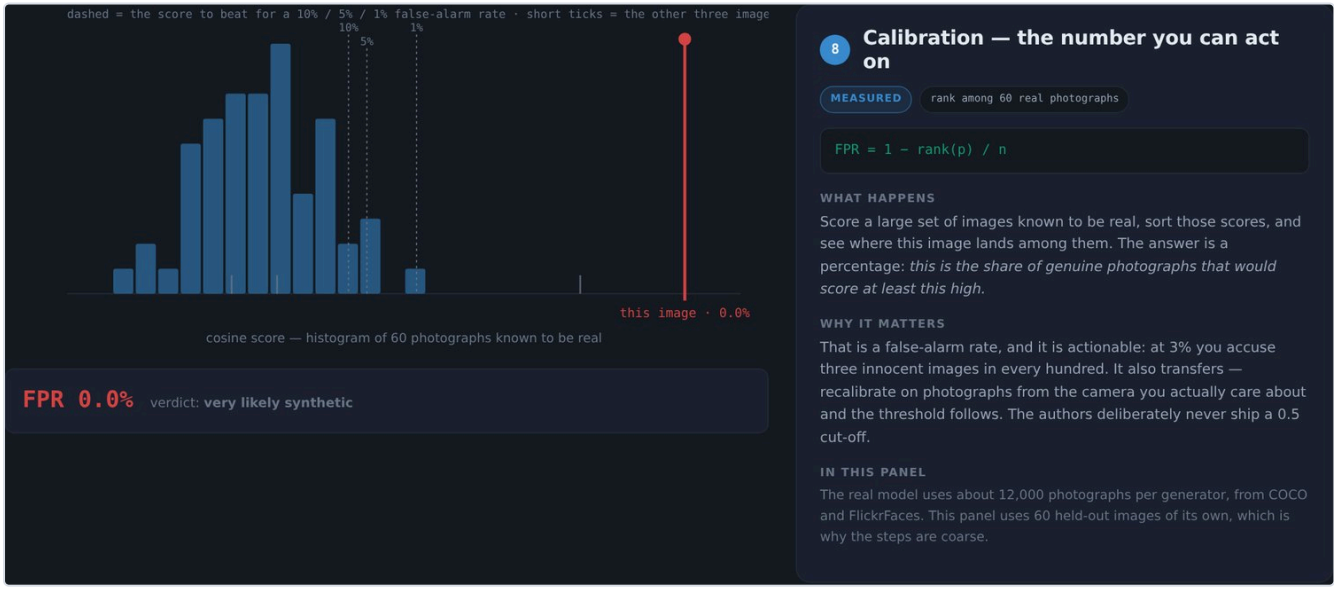


Figure 16. Stage 8. The histogram is the distribution of scores from 60 photographs known to be real. The coloured line is your image. Its false-positive rate is simply its rank in this distribution.

A raw score of 0.095 means nothing on its own. The detector ranks it against a large set of images known to be real and reports *"this is the share of genuine photographs that would score at least this high"*. That is a false-alarm rate, and it is actionable: at 3 % you accuse three innocent images in every hundred.

It also transfers. Recalibrate on photographs from the camera you actually care about and the threshold follows automatically. This is why the authors deliberately ship no 0.5 cut-off — and it is one of the more quietly sophisticated decisions in the paper.

11.8 The live numbers strip

Four cells beneath the stage navigator — cosine, score p, false-positive rate, verdict — fill in progressively as you advance through the stages. Before the relevant stage each reads “—”. This makes the dependency chain visible: you cannot have a probability before you have a cosine, and you cannot have a false-positive rate before you have a probability.

12 · The closing panels

12.1 The real model in numbers

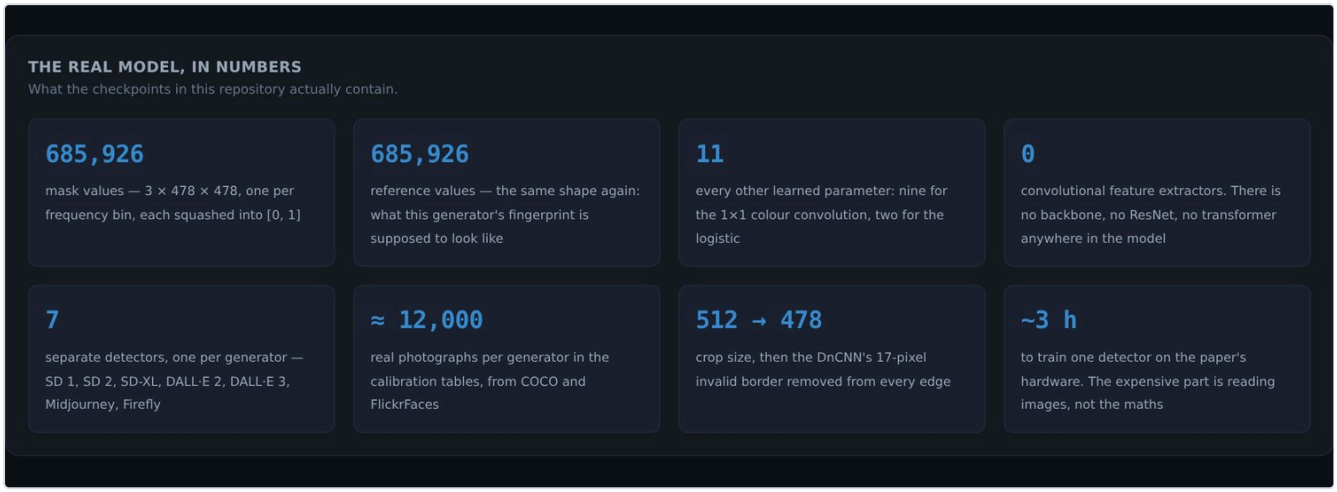


Figure 17. Eight statistics about what the checkpoints in the repository actually contain.

Eight tiles giving the quantitative shape of the trained model. Three of them are worth saying out loud:

- **685,926 mask values and 685,926 reference values** — 3 × 478 × 478 each. These are essentially the entire model.
- **11 other parameters** — nine for the 1×1 colour convolution, two for the logistic. That is all.
- **Zero convolutional feature extractors.** There is no backbone, no ResNet and no transformer anywhere in the model. This is what lets it train on a few hundred images rather than a few hundred thousand, and it is why the mask can simply be looked at.

12.2 What it cannot do

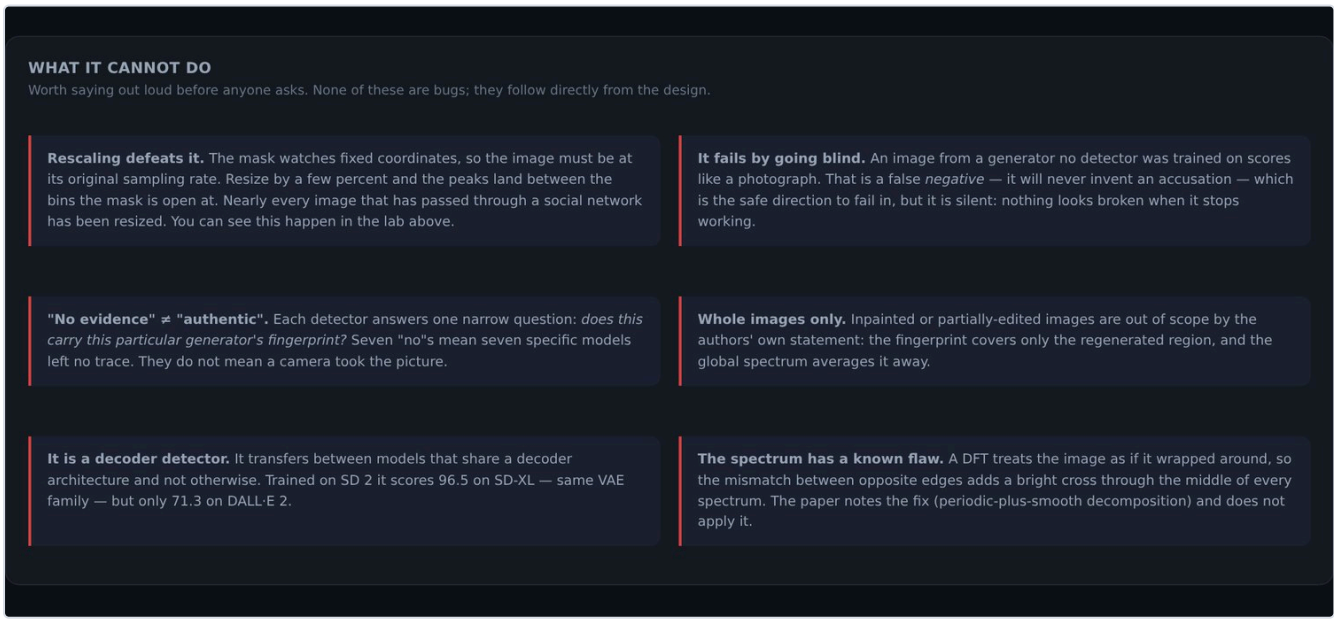


Figure 18. Six limitations, all of them consequences of the design rather than defects in the implementation.

LIMITATION	WHY IT FOLLOWS FROM THE DESIGN
Rescaling defeats it	The mask watches fixed coordinates, so the image must be at its original sampling rate. Resampling by a factor *s* scales every frequency by 1/*s*, and the peaks land between the bins the mask has open. Nearly every image that has passed through a social network has been resized. The lab demonstrates this directly.
It fails by going blind	An image from an unseen generator scores like a photograph — a false *negative*. That is the safe direction to fail in, but it is silent: nothing looks broken when it stops working.
“No evidence” ≠ authentic	Seven narrow questions, seven “no” answers. Silence, not an alibi.
Whole images only	Inpainted or partially edited images are out of scope by the authors' own statement: the fingerprint covers only the regenerated region and the global spectrum averages it away.
It is a decoder detector	It transfers between models that share a decoder architecture and not otherwise. Trained on SD 2 it scores 96.5 on SD-XL — the same VAE family — but only 71.3 on DALL·E 2.
The spectrum has a known flaw	A DFT treats the image as if it wrapped around, so the mismatch between opposite edges adds a bright cross through the middle of every spectrum. The paper names the fix — periodic-plus-smooth decomposition, Moisan 2011 — and does not apply it.

> **You can see that flaw on the page**
>
> Look at any spectrum in Step 2 or Step 7: there is a bright vertical and horizontal cross through the centre. That is not the image and it is not a generator. It is the discontinuity between the left edge and the right edge of the crop, which the DFT interprets as a genuine high-frequency signal. Naming this unprompted is a good way to show you understand what you are looking at.

Appendix A · All the mathematics in one place

A.1 The two-dimensional DFT

$$F[k_y, k_x] = \sum_y \sum_x f[y, x] \cdot \exp\left(-2\pi i \left(\frac{k_y y}{N} + \frac{k_x x}{N} \right) \right)$$

Computed separably: a 1-D FFT along every row, then a 1-D FFT down every column. The implementation is an iterative radix-2 Cooley–Tukey transform with bit-reversal permutation, giving $O(N^2 \log N)$ for an $N \times N$ image. The inverse is the same routine with the sign of the exponent flipped and a division by N.

A.2 fftshift

$$\text{shifted}[k] = \text{original}[\,(k + N/2) \bmod N\,]$$

A cosmetic re-indexing that moves the zero-frequency term from the corner to the centre, so that distance from the middle of the displayed picture means frequency magnitude. Applied to every spectrum shown on the page.

A.3 Log magnitude

$$s[k] = \log\left(|F[k]| + \varepsilon \right) \qquad \varepsilon = 10^{-8}$$

The magnitude alone spans many orders of magnitude between DC and the high frequencies; without the logarithm every spectrum would render as a single white pixel on black. The ε prevents $\log(0)$.

A.4 Amplitude of a single bin, in grey levels

$$A_k = 255 \cdot 2 \cdot \frac{|F[k]|}{N^2}$$

The factor 2 accounts for the conjugate pair. This is the number reported by the *"amplitude in the image"* readout in Step 2.

A.5 The mask

$$\text{real model:} \quad M = \sigma\left(\text{boxblur}_{3\times3}(\theta) \right), \quad \theta \text{ learned by gradient descent}$$

$$\text{the lab:} \quad z[k] = \frac{|\text{mean}_{\text{fake}}[k] - \text{mean}_{\text{real}}[k]|}{\text{pooled_std}[k]}$$

$$M[k] = \sigma\left((z[k] - \text{threshold}) \cdot 9 \right)$$

In both cases M lies in [0,1], holds one number per frequency bin, and is completely static: the same map multiplies every image the detector ever sees.

A.6 The score

```
v     =  M ⊙ s                      // mask the spectrum, element-wise
v̂     =  ( v − mean(v) ) / std(v)   // BatchNorm in the real model
cos   =  ⟨ v̂ , R ⟩ / ( ‖v̂‖ ‖R‖ )    // R is the learned reference
p     =  σ( e^a · cos + b )         // clf_a, clf_b in the checkpoint
```

The similarity map displayed on both Step 6 and Step 7 is the element-wise product $\hat{v}_k \cdot R_k / (\|\hat{v}\|\|R\|)$ — every pixel is one term of that sum, so the whole map adds up to the cosine.

A.7 Calibration

```
FPR( p )  =  1  −  rank( p ) / n        n = size of the known-real set
```

where rank is found by binary search in the sorted array of scores from images known to be real. The real detectors use roughly 12,000 photographs each, drawn from COCO and FlickrFaces. The lab uses 60 of its own, which is why its steps are visibly coarse.

A.8 The generic model

Worth knowing because it is easy to get wrong. The overall verdict is *not* the maximum of the seven raw scores. It is a two-stage procedure: each raw score is first converted to a calibrated score against its own table; the maximum of those seven calibrated scores is then taken; and that maximum is calibrated a second time against a separate table (`all_to_max.csv`). The second calibration is what corrects for the fact that taking a maximum over seven detectors inflates the score of every image, including genuine ones.

A.9 The verdict bands

FPR	LABEL	READING
< 1 %	very likely synthetic	Fewer than 1 in 100 genuine photographs score this high.
< 5 %	likely synthetic	Fewer than 1 in 20.
< 10 %	possibly synthetic	Fewer than 1 in 10.
≥ 10 %	no evidence	Too common among real photographs to mean anything.

Appendix B · Control reference

Every interactive element on the page, what it changes, and the computation it triggers. Element identifiers are given so the behaviour can be traced in the source.

STEP	CONTROL	RANGE / DEFAULT	WHAT HAPPENS ON CHANGE
Source bar	**Upload an image** `#srcPick`	–	POSTs the file to `/api/source`; on reply, reloads the tile, recomputes both cached spectra, redraws Steps 2–5, and retrains the lab. Polls for the detector verdict.
Source bar	**Use the sample photo** `#srcSample`	–	DELETEs the stored image and reverts to the bundled cameraman tile.
Step 1	**k_y** `#ky`	0–60, 0	Regenerates the cosine grating, FFTs it, applies a 5×5 dilation, redraws both canvases.
Step 1	**k_x** `#kx`	0–60, 6	As above.
Step 2	**Pointer move** `#specC`	–	Converts screen position to a bin; redraws the cached spectrum plus overlay, regenerates the single-frequency pattern, updates four readouts, the caption, the adaptive note, and the marker on the radial chart.
Step 2	**Click**	–	Locks or unlocks the probe. Crosshair turns blue; readout says “locked”.
Step 2	**Spectrum tabs** `#specTabs`	image / residual	Swaps which cached analysis is displayed. Instantaneous — both were computed at load. Also changes the middle panel to the matching source signal.
Step 3	**Coefficients kept** `#keep`	0.01–10 %, 1.2 %	Sorts the magnitudes, thresholds, zeroes the rest, runs an inverse FFT, redraws the reconstruction and the keep-map, updates the caption with the absolute count.
Step 4	**Cut radius** `#rad`	3–120, 18	Splits the spectrum at the radius, runs *two* inverse FFTs, redraws the low-pass and high-pass reconstructions and the radius map.
Step 5	**Artifact amplitude** `#amp`	0–6.0/255, 1.2/255	Rebuilds the modified image, the amplified difference and the spectrum; updates the caption with the largest per-pixel change.
Step 5	**Artifact frequency** `#afreq`	8–120, 64	As above, moving the peaks outwards along a fixed line.
Step 5	**Blind test** `#blindBtn`	toggle	Randomises the panel order, relabels them A and B, blurs the difference and spectrum panels. Pressing again reveals which one carried the artifact.
Step 6	**Stage thumbnails**	4 stages	Selects which of the real pipeline stages is explained underneath.
Step 6	**Detector tabs**	7 detectors	Switches to that detector's maps and its four numbers.
Step 6	**Layer tabs**	3 layers	Similarity map / trained mask / masked spectrum. All three were rendered server-side and sent with the result, so switching is

			instant.
Step 7	**Source buttons** `#srcRow`	`4 images`	Selects which of the four test images the stages display. All four are always scored, so the summary chart never changes.
Step 7	**Fingerprint strength** `#strength`	`0–4.5/255, 1.5/255`	Rebuilds the three planted images (including a fresh JPEG round-trip), rescores all four, redraws everything. Debounced by 90 ms so dragging stays smooth. **Does not retrain the detector.**
Step 7	**Back / Next**	`stages 1–8`	Moves through the stages. The left and right arrow keys do the same.
Step 7	**Stage chips**	`8 chips`	Jump straight to any stage.
Step 7	**Play**	`4.2 s per stage`	Advances automatically and stops at the last stage. Any manual interaction cancels it.

> **A note on responsiveness**
>
> Everything in Steps 1 to 5 recomputes a full 256×256 FFT — or in Step 4 two of them — on every single slider movement, and still feels continuous. That is the practical payoff of the Cooley–Tukey algorithm: roughly half a million operations rather than the four billion a naïve DFT would need.

Appendix C · What is real and what is simulated

This table exists so that nothing on the page can be mistaken for a stronger claim than it makes. If one page of this document is worth memorising before a viva, it is this one.

PART OF THE PAGE	STATUS	EXACTLY WHAT IT IS
Steps 1–5 spectra, reconstructions, probe readouts	**Real measurements**	Genuine Fourier analysis of the actual image, computed in the browser. Every number is a real property of the picture. Nothing is illustrative.
Step 5 the planted artifact	**Synthetic, by design**	The page adds the artifact itself. It demonstrates that such an artifact is detectable; it does not claim any generator produced it.
Step 6 the verdict and all its maps	**The real trained model**	Seven checkpoints from the paper's authors, the real DnCNN, the real masks, the real calibration tables. Identical to what `detect_one_image.py` prints. This is the only part that makes a claim about your image.
Step 7 the lab detector	**A real detector, on synthetic data**	The mask, reference, logistic and calibration table are genuinely fitted — but from a dataset the page manufactures, with a fingerprint of its own invention. A faithful miniature of the method, not a verdict on the picture.

C.1 How the lab differs from the published model

ASPECT	THE PAPER	THE LAB
Input	3 × 512 × 512, YCbCr	1 × 256 × 256, luma only
Denoiser	17-layer DnCNN, pre-trained, frozen	3 × 3 box high-pass
Mask fitting	20 epochs of gradient descent, binary cross-entropy	Closed form: a per-bin z-score of the class difference
Colour mixing	A 1 × 1 convolution, nine learned weights	Not applicable — one channel
Normalisation	BatchNorm with running statistics	Explicit standardisation of the masked vector
Training set	Hundreds of images per generator	28 variants of your image
Calibration set	≈ 12,000 real photographs	60 held-out variants

The important line is the third. A closed-form z-score and twenty epochs of gradient descent are computing the same thing — a per-bin measure of how reliably the two classes separate — one directly, the other by optimisation. That is why the lab is a fair miniature rather than a cartoon.

C.2 Where the numbers on the detect page come from

`analysis.py` is a wrapper, not a reimplementation. It calls the repository's own code for every quantity it displays: `MaskSimDataset.process()` for the preprocessing, `MaskSim.compute_probs()` for the score, `MaskSim.compute_similarity_map()` for the maps, the same concatenated COCO + FlickrFaces CSVs for calibration, and the same two-stage generic model. If the web page and `detect_one_image.py` ever disagree, the wrapper is what is wrong.

> **One optimisation, declared**
>
> The DnCNN is the expensive part, and running it once per detector means fourteen passes per image. At start-up the engine *fingerprints each checkpoint's denoiser weights*; only if all seven carry an identical frozen denoiser does it compute the residual once and reuse it — roughly a minute reduced to a few seconds on CPU. If the fingerprints differ it falls back to the slow, always-correct path. The start-up log states which: `denoiser shared across detectors: True/False`.

Appendix D · A six-minute demonstration script

A running order that builds the argument rather than touring the interface. Timings are generous; the whole thing fits comfortably in six minutes with pauses.

0:00 — Frame the problem (30 s, no clicking)

“Every image generator finishes by upsampling a small internal picture to full size. The filters that do it are learned weights, and they are the same weights for every image that model makes — so every output carries a faint repeating residue, well under one grey level. You cannot see it. The question is whether you can measure it.”

0:30 — Step 1 (45 s)

Drag k_x up and down: “distance from the centre is how fast the stripes repeat.” Set k_y to 9: “direction is which way they run.” Set both to zero: “and the middle is just the average brightness. That is the whole vocabulary.”

1:15 — Step 2 (75 s — the core of the demonstration)

Hover near the centre: read out the amplitude — tens of grey levels. Hover out at the edge: read it again — thousandths. “Four orders of magnitude between the picture and the place a fingerprint would live.” Then click the **residual** tab. “That is what the detector actually computes. The scene is gone and the spectrum is nearly flat — which is the only condition under which something a thousandth of a grey level deep has a chance.”

2:30 — Step 5 (60 s — the moment that lands)

Press the blind-test button first. “One of these two has a fingerprint planted in it, just over one grey level deep. Which?” Let them guess. Reveal, then point at the fourth panel. “Invisible in the picture. Unmistakable in the spectrum. That is the entire method in one screen.”

3:30 — Step 7, stage 4 (60 s)

Jump straight to the stage chip labelled **The mask**. Point at the first two panels: “the average spectrum of a hundred real images, and of a hundred fakes. Identical to the eye.” Point at the third: “divide their difference by its own variability and ten dots appear out of nothing. That is what the mask learns, and that map is the whole model — no backbone, no ResNet, 686,000 numbers you can simply look at.”

4:30 — Step 7, the resized source (45 s — the honest ending)

Click through the four sources in order and read the summary chart. “Untouched: nothing. Planted: caught instantly. JPEG at quality 60: still caught, because the artifact lives at frequencies JPEG keeps. Resized by a quarter: back to looking like a photograph. Rescaling slides every peak off the bins the mask is watching, and the detector goes silent with no sign that it has failed. That is the real limitation of this family of methods, and it is why it is not deployed as a public verdict service.”

5:15 — Step 6 (45 s, if you have an image to show)

Upload a generated image, wait for the verdict, and walk the similarity map. “These are the real trained checkpoints from the paper, not my miniature. And note what the tool says when it finds nothing: no evidence — which is silence, not an alibi.”

Have this ready before you start

One AI-generated image at least 512×512, saved as PNG and never resized or re-uploaded anywhere. If it has been through a messaging app it will have been recompressed and rescaled, and the demonstration will fail for exactly the reason you are about to explain — which is either a disaster or the best possible teaching moment, depending on whether you saw it coming.

Appendix E · Questions you are likely to be asked

“Why the Fourier transform rather than a CNN?”

Because the artifact is *periodic and global*, and the DFT is the transform that concentrates periodic global energy into single coefficients. A CNN would have to learn that concentration from data. Doing it analytically means the model needs only one weight per frequency, which is why it trains on hundreds of images instead of hundreds of thousands, and why you can inspect what it learned.

“Why discard the phase?”

Because *where* the periodic pattern sits in the image is irrelevant — a fingerprint that starts at a different offset is the same fingerprint. Magnitude is translation-invariant, so discarding phase makes the feature invariant to exactly the nuisance you want to ignore. It also halves the representation.

“Why a cosine similarity rather than a Euclidean distance?”

Because a cosine ignores scale. Two images of the same scene at different exposures or noise levels have masked spectra that differ by a multiplicative factor; a distance would call them different, a cosine calls them identical. Only the pattern counts.

“Why not just threshold at 0.5?”

Because the number that comes out of the logistic only looks like a probability. It was fitted on one training set with a 50/50 class balance. Change the balance, the camera or the compression and 0.9 no longer means 90 %. Ranking the score against a set of known-real images instead gives a false-alarm rate, which is a quantity you can set policy against and which recalibrates cleanly to a new domain.

“What does the bright cross through every spectrum mean?”

It is the DFT's implicit assumption that the image wraps around. The left edge does not match the right edge, so the transform sees a step discontinuity and reports strong energy along both axes. The paper names the standard fix — periodic-plus-smooth decomposition — and chooses not to apply it, which is a fair thing to criticise.

“What happens on an image from a generator you did not train on?”

It scores like a photograph. The failure is a false negative, which is the safe direction, but it is silent. This is the fundamental limitation of a per-generator fingerprint approach and the page says so in three separate places.

“Could someone remove the fingerprint deliberately?”

Easily, and that is important to concede. Resizing by a few percent does it, as the lab demonstrates. So does adding noise at the right frequencies, or a mild low-pass. The method is designed for unmodified generator output, not for an adversary.

“How much of this did you build?”

The honest answer: the seven trained checkpoints, the calibration tables and the core model code are the paper authors' work. The entire interface, the browser-side Fourier analysis, the spectrum probe, the artifact demonstration and the miniature detector in Step 7 were built for this project — the last of these is a working reimplementation of the method from scratch, in about four hundred lines, fitted and calibrated live in the browser.

“What would you do next?”

Four things, in order of value: a quantitative evaluation with ROC curves and a seven-by-seven cross-generator matrix; a robustness sweep measuring AUC against JPEG quality and resize factor; ablations removing the denoiser, the mask

and the logarithm one at a time; and implementing the periodic-plus-smooth decomposition the paper declined to apply, to measure whether it helps.

Reference. Y. Li, Q. Bammey, M. Gardella, T. Nikoukhah, J.-M. Morel, M. Colom and R. Grompone von Gioi, *MaskSim: Detection of synthetic images by masked spectrum similarity analysis*, IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2024. Fallback test image: the standard *cameraman* tile (MIT, distributed with scikit-image).